MediCore

Plataforma hospitalaria: web del personal (Razor Pages), API (ASP.NET Core) y app móvil del paciente (.NET MAUI, Android). Proyecto 2 del Proyecto Final de QA. **Estado: aplicación, pruebas y pipeline completos en QA local.** Propuesta actual: NO-GO para producción y GO condicionado para QA (informe final).

Requisitos

Herramienta	Para qué	Versión
.NET SDK	compilar y ejecutar (hoy)	10.0.x
Git	control de versiones	cualquiera reciente
Visual Studio 2022 17.14+ o 2026	IDE (soporta `.slnx`)	opcional
Docker Desktop	ambiente QA, pruebas y pipeline	necesario
Workload MAUI + Android SDK	app móvil y Maestro	opcional
Terraform, Ansible	infraestructura	opcional; la validación y la simulación corren en contenedores

Ejecutar

Primera vez (dependencias y base de datos):

```
cp .env.example .env                # cambia las contraseñas; .env no se versiona
docker compose up -d                # PostgreSQL (puerto 5433), Redis y RabbitMQ (panel en
http://localhost:15672)
dotnet tool restore                 # instala dotnet-ef (herramienta local del repositorio)
dotnet user-secrets set "ConnectionStrings:Postgres"
"Host=localhost;Port=5433;Database=medicore;Username=medicore;Password=<POSTGRES_PASSWORD de tu .env>" -
-project src/MediCore.Api
```

Aplicar migraciones (`database update` lee la cadena de la variable de entorno; ejemplo en PowerShell):

```
$env:ConnectionStrings__Postgres =
"Host=localhost;Port=5433;Database=medicore;Username=medicore;Password=<POSTGRES_PASSWORD>"
dotnet ef database update --project src/MediCore.Infrastructure --startup-project src/MediCore.Api
```

Secretos locales de la API (inventa valores propios; nunca se versionan):

```
dotnet user-secrets set "ConnectionStrings:Redis" "127.0.0.1:6379,password=<REDIS_PASSWORD de tu .env>"
--project src/MediCore.Api
dotnet user-secrets set "ConnectionStrings:RabbitMq" "amqp://medicore:<RABBITMQ_DEFAULT_PASS de tu
.env>@127.0.0.1:5672/" --project src/MediCore.Api
dotnet user-secrets set "Jwt:SigningKey" "<texto aleatorio de 32 caracteres o más>" --project
src/MediCore.Api
dotnet user-secrets set "Seed:DevPassword" "<contraseña de desarrollo para los usuarios ficticios>" --
project src/MediCore.Api
```

Usa `127.0.0.1` y no `localhost` para Redis: Docker publica el puerto solo en IPv4 y el cliente puede intentar IPv6 primero.

Al arrancar la API en Development se aplican las migraciones y, si la tabla de usuarios está vacía, se crean usuarios ficticios con la contraseña de `Seed:DevPassword`: `elena.vargas@example.com` (Recepción), `lucia.mendez@example.com` (Médico), `daniela.reyes@example.com` (Laboratorio), `admin@example.com` (Administrador) y `ana.garcia@example.com` (Paciente, solo móvil). También se crean 4 pacientes y 3 médicos ficticios del diseño (Ana García queda vinculada a su cuenta y la Dra. Lucía Méndez a la suya). Cada médico recibe un horario de lunes a viernes de 08:00 a 12:00 y la Dra. Méndez cuatro citas el próximo día hábil, como en la agenda del diseño.

El puerto 5433 evita choques con un PostgreSQL local en 5432 (cámbialo con `POSTGRES_PORT`).

Después:

Abre `MediCore.slnx` en Visual Studio, o desde la terminal en la raíz (la web necesita la API en marcha para iniciar sesión):

```
dotnet build MediCore.slnx
dotnet test MediCore.slnx             # las de integración levantan PostgreSQL y Redis con
Testcontainers (requiere Docker)
dotnet run --project src/MediCore.Api --launch-profile http   # http://localhost:5011/api/health  (y
/api/health/ready comprueba PostgreSQL)
dotnet run --project src/MediCore.Web --launch-profile http   # http://localhost:5298
```

Con la API en Development, el contrato OpenAPI está en `http://localhost:5011/openapi/v1.json`.

Expediente, consultas y recetas

- El médico abre el expediente desde su agenda o desde "Pacientes autorizados". Cada consulta pertenece a una cita (una consulta por cita; de hoy o anteriores) y cada receta a una consulta, con 1 a 10 medicamentos.
- **Expediente compartido:** un médico con relación de atención válida con el paciente (una cita no cancelada o una consulta registrada) lee las consultas y recetas que escribieron **otros médicos**, identificadas por autor. **Nadie modifica registros de otro profesional** (solo el autor crea recetas sobre su consulta; las consultas y recetas no se editan ni se borran). Sin relación válida no hay acceso. Recepción, Administrador y Laboratorio no acceden a la información clínica.
- El paciente lee únicamente sus propias recetas, de cualquiera de sus médicos (`GET /api/prescriptions/me`, para la app móvil).
- La pestaña Laboratorio del expediente lista los resultados del paciente (ver la sección siguiente).
- La auditoría y las bitácoras registran quién accedió a qué recurso y cuándo, nunca el contenido clínico.

Laboratorio y notificaciones (entrega 6)

- **Laboratorio** registra resultados (WEB-17: paciente, tipo de examen, fecha y de 1 a 10 parámetros con valor, unidad y rango), los lista con el estado de la notificación (WEB-16) y ve su detalle (WEB-18). Los resultados no se editan ni se borran: una corrección es un resultado nuevo.
- **Quién ve qué:** Laboratorio, todos los resultados; el **Médico**, los de pacientes con quienes tiene una relación válida de atención (desde la pestaña Laboratorio del expediente); el **Paciente**, solo los suyos (`GET /api/labs/results/me`, para la app móvil). **Recepción y Administrador no tienen acceso clínico.** Un acceso ajeno devuelve 404 (no confirma que exista) y queda auditado; la auditoría y las bitácoras no guardan tipo de examen, valores ni observaciones.
- **Notificación asíncrona (RabbitMQ real, outbox transaccional):** al guardar un resultado se guarda en la **misma transacción de PostgreSQL** el evento `LabResultReady` (solo identificadores). Un proceso en segundo plano lo publica en RabbitMQ y un consumidor crea la notificación interna del paciente. Si RabbitMQ no está disponible, el resultado queda guardado (estado "Pendiente"), el evento espera con reintentos crecientes y se publica cuando el broker vuelve. El consumidor es idempotente (un mensaje reentregado no duplica la notificación), reintenta los fallos con espera y, agotados los intentos, deja el mensaje en la cola de mensajes fallidos (`medicore.notifications.dlq`). En WEB-18, Laboratorio puede **reintentar la notificación** si sigue pendiente.
- **API para la futura app móvil:** `GET /api/notifications` (paginado, `unreadOnly`), `GET /api/notifications/unread-count` y `POST /api/notifications/{id}/read`. No hay portal web del paciente ni se envían correos.
- **Configuración:** `ConnectionStrings:RabbitMq` en user-secrets (arriba) y la sección `Messaging` de `appsettings.json` (intervalos, intentos). `Messaging:Enabled=false` apaga publicador y consumidor.
- **Evidencia reproducible:** `python scripts/evidencia_laboratorio.py` (con la API en marcha y `MEDICORE_PASSWORD`) ejecuta los escenarios con el broker real, incluido detener y volver a iniciar el servicio `rabbitmq` de compose, y escribe `documentacion/evidencias/entrega-6-laboratorio.md`.

Pruebas en equipos con Smart App Control

Windows con Smart App Control en modo activo bloquea la carga de los `.dll` sin firma que compila el proyecto (migraciones, pruebas, API). Sin desactivarlo, `scripts/dotnet-linux.sh` y `scripts/dev-stack-linux.sh` ejecutan el mismo código en contenedores Linux del SDK oficial (se descarga `mcr.microsoft.com/dotnet/sdk:10.0`; la caché de NuGet vive en el volumen nuevo `medicore-nuget`). Los volúmenes y la red de compose (`medicore_*`) no se modifican. Detalle en `CLAUDE.md`.

App móvil del paciente (entrega 7)

App Android con .NET MAUI (`src/MediCore.Mobile`) sobre la API real: inicio y cierre de sesión, próxima cita, mis citas y detalle, reserva en tres pasos, cancelación, resultados, recetas y notificaciones (MOB-01 a MOB-12). La lógica está en `src/MediCore.Mobile.Core` y se prueba sin Android. **Cómo compilar, instalar, conectar con la API en Docker y ejecutar Maestro: documentacion/05-app-movil.md.** La app se compila con `scripts/android-build-linux.sh` (contenedor Linux) y se prueba en un emulador con Maestro; las herramientas (SDK de Android, emulador, Maestro) ya están instaladas en este equipo. Pasos exactos en el documento.

Contenedores y ambiente QA (entrega 8)

El sistema completo corre en contenedores (API, web, proceso de mensajería, PostgreSQL, Redis, RabbitMQ y proxy inverso). **Guía, arquitectura, costos estimados de Azure y procedimientos de crear, modificar y destruir: documentacion/06-infraestructura.md.**

```
scripts/qa.sh init      # .env.qa con contraseñas aleatorias (no se versiona)
scripts/qa.sh up        # construye, migra y levanta; solo el proxy publica puertos
(http://127.0.0.1:8080)
scripts/qa.sh seed      # usuarios y datos ficticios
scripts/qa.sh smoke     # pruebas de humo de web y API
scripts/qa.sh down      # conserva los volúmenes; "destroy" los borra (pide confirmación)
scripts/qa.sh backups   # lista los respaldos automáticos; también: backup, export, import, restore
```

La infraestructura de Azure (Terraform y Ansible) se **ejecutó sin costo, no en Azure**: `scripts/iac-simulacion.sh` usa un proveedor simulado y un Ubuntu desechable. No se creó ningún recurso real; el plan y su costo esperan aprobación (sección 3 del documento). La app Android es un APK, no un servicio del Compose.

Agenda y citas

- Las horas son del reloj de la clínica (`Clinic:TimeZone`, por defecto `America/Guatemala`); se puede reservar desde hoy hasta `Scheduling:MaxDaysAhead` días (90). Las citas duran 30 minutos.
- La disponibilidad se cachea en Redis (`Cache:AvailabilityTtl`, 2 min) y se invalida al reservar, cancelar o cambiar el horario del médico.
- Para reintentar sin duplicar, `POST /api/appointments` acepta el encabezado `Idempotency-Key` (8 a 64 caracteres, letras, números, `-` y `_`). La web lo genera en cada formulario de reserva.
- Una cita no se puede solapar con otra activa del mismo médico ni del mismo paciente: lo garantiza PostgreSQL (extensión `btree_gist`, que incluye la imagen oficial de Docker; en Azure Database for PostgreSQL debe habilitarse en la lista de extensiones permitidas).

Estructura

```
MediCore.slnx
docker-compose.yml      dependencias locales
src/MediCore.Api        Web API
src/MediCore.Contracts  contratos compartidos API/Web/móvil (DTO, roles)
src/MediCore.Domain     entidades y reglas (sin dependencias)
src/MediCore.Infrastructure  EF Core, PostgreSQL, Redis, migraciones
src/MediCore.Web        Razor Pages del personal
tests/MediCore.Api.Tests   integración (Testcontainers)
tests/MediCore.UnitTests   reglas de dominio
documentacion/          alcance, plan de trabajo, trazabilidad
Docs/  Referencias/     documento del proyecto y diseño aprobado (solo lectura)
```

Estado de las pruebas

Suite	Resultado
xUnit	304/304 (48 unitarias, 50 móviles, 206 de integración)
Insomnia	65/65 comprobaciones en 61 solicitudes
Cypress en Chrome	116/116, incluidas 14 pantallas con axe
Maestro	8/8 con el código final, en emulador contra QA
k6	Load, Stress y Spike dentro de umbrales, en equipo local
SonarQube	Puerta aprobada, 0 bugs, 74,2 % de cobertura
ZAP	0 alertas altas en web y API con 5 roles; 40 de 45 operaciones alcanzadas
Fortify	Sin ejecutar: falta licencia
Usabilidad	Definida, sin aplicar a personas

Documentos de entrega

Documento	Contenido
Plan de pruebas	Las 20 secciones exigidas
Matriz de riesgos	Confidencialidad, integridad y disponibilidad
Casos de prueba	Casos críticos con su evidencia
Registro de defectos	Pasos, severidad, prioridad y evidencia
Informe final	Métricas y decisión GO / NO-GO
Guion de defensa	Demostraciones en vivo
Seguridad y accesibilidad	Hallazgos y su tratamiento

Pipeline

Jenkins y SonarQube corren en contenedores:

```
docker compose -p medicore-ci --env-file ci/.env.ci -f ci/docker-compose.ci.yml up -d --build
```

Jenkins queda en `http://127.0.0.1:8090` y SonarQube en `http://127.0.0.1:9000`. `ci/.env.ci` no se versiona y define `JENKINS_ADMIN_PASSWORD`, `SONAR_ADMIN_PASSWORD`, `SONAR_TOKEN` y `MEDICORE_BRANCH`. Para crear la puerta de calidad, usa `scripts/sonar-setup.sh`.

Cada corrida borra los datos de QA y los vuelve a sembrar. Solo afecta los volúmenes `medicore-qa_*`.

Seguridad

No se guardan secretos en el repositorio. Copia `.env.example` a `.env` (ignorado por Git) cuando una entrega lo requiera. Todos los datos de ejemplo son ficticios.